

La ventana de escritorio

AlgoProject — manual de uso. Generado el 2026-09-26 con `tools/manual.py`.

1. 35. La aplicación de escritorio — plantillas
2. 38. La aplicación de escritorio — activos
3. 48. La aplicación de escritorio — generación, o por dónde va el proyecto
4. 44. La aplicación de escritorio — estrategias
5. 45. La aplicación de escritorio — la puerta IS/OOS

35. La aplicación de escritorio — plantillas

La ventana. No es una pestaña del navegador ni un panel más: es la primera rebanada de la aplicación que va a envolver SQX y Python, y por ahora contiene un solo módulo, el de las **plantillas**.

Qué pregunta responde

Tres, y las tres se contestaban hasta hoy abriendo dos CSV en Excel y una carpeta en el explorador:

- ¿Qué plantillas tengo, de qué tipo, y en qué mercados y timeframes las he probado?
- ¿De qué tengo poco? Qué arquetipo no he tocado, qué timeframe está vacío, qué idea probé una sola vez y no volví a mirar.
- ¿Cómo empiezo una plantilla nueva sin olvidarme de ninguna de las preguntas? La ventana te entrevista, escribe el borrador del brief y te deja el comando listo para pegar en Claude Code.

Cuándo lo usas, y cuándo no

Lo usas para mirar la librería, para decidir qué construir a continuación, para anotar el veredicto de una corrida en cuanto lo tengas, y para redactar la idea de una plantilla nueva.

No lo usas para construir nada. La ventana no arranca SQX, no instala bloques y no emite ningún .sqx . Todo eso lo hace /sqx-strategy-template y /template-run , que escriben en instalaciones de verdad y son el carril del conductor y el del custodio. Una ventana que escribiera en SQX_w1 por su cuenta rompería la tabla de carriles de sqx/CLAUDE.md en cuanto hubiera dos sesiones abiertas.

Tampoco es donde se editan los CSV. Escribe tres cosas y solo tres: el estado de una plantilla, su nota, y el veredicto de una corrida.

Antes de empezar

- pip install -r requirements.txt — la app añade PySide6, FastAPI, uvicorn y httpx.
- Nada más. No hace falta SQX abierto ni cerrado, no hace falta ningún worker arrancado, y no toca ninguna instalación. Si ~/Desktop/AlgoData/templates/ no existe todavía, la ventana abre vacía en vez de fallar.

Cómo se ejecuta

Tres formas, las tres hacen exactamente lo mismo:

dónde	qué haces
el escritorio	doble clic en el icono AlgoProject
el menú de aplicaciones	buscas AlgoProject
cualquier terminal, desde cualquier carpeta	algoui

```
algoui
```

Las tres acaban en bin/algoui , que no hace más que llamar a python3 -m ui.desktop.launch desde la raíz del proyecto. El icono es la propia matriz de cobertura en miniatura, y es el mismo que sale en la barra de tareas y en el alt-tab.

algoui funciona desde cualquier sitio porque ~/.local/bin/algoui es un enlace al del repositorio y el script resuelve el enlace antes de buscar la raíz. Si alguna vez mueves la carpeta del proyecto, rehaz el enlace y la línea Exec= de ~/.local/share/applications/algoproject.desktop : las dos llevan la ruta

escrita.

En Windows la ventana funciona igual, sin SQX: doble clic en `bin\algoui.cmd`, o `bin\algoui.cmd` desde una consola. Hace lo mismo que `bin/algoui` y además activa el modo UTF-8 de Python, sin el cual los textos en español se leen en cp1252. Lo único que cambia sin SQX es la ficha de una plantilla: sus huecos atados a un grupo salen como «*atado a un grupo que esta instalación no tiene*», porque los nombres de los grupos viven en el `blockGroups.xml` de la instalación. Cómo montarla en un PC nuevo está en el capítulo de GitHub.

flag	obligatorio	qué hace
—	—	no tiene flags: el puerto sale de <code>ui_port</code> en <code>config/machine.yaml</code> , 8765 por defecto

Si algo va mal y la ventana no abre, el demonio se arranca por separado para ver su error:

```
python3 -m ui.daemon.serve
```

Dos procesos, no uno. `bin/algoui` levanta el demonio si no está ya escuchando y abre la ventana. Al cerrar la ventana, el demonio se apaga **solo si lo arrancó ella**: dos ventanas comparten uno, y matarlo le quitaría la librería a la otra. El demonio escucha en 127.0.0.1 y en ningún sitio más.

Qué produce

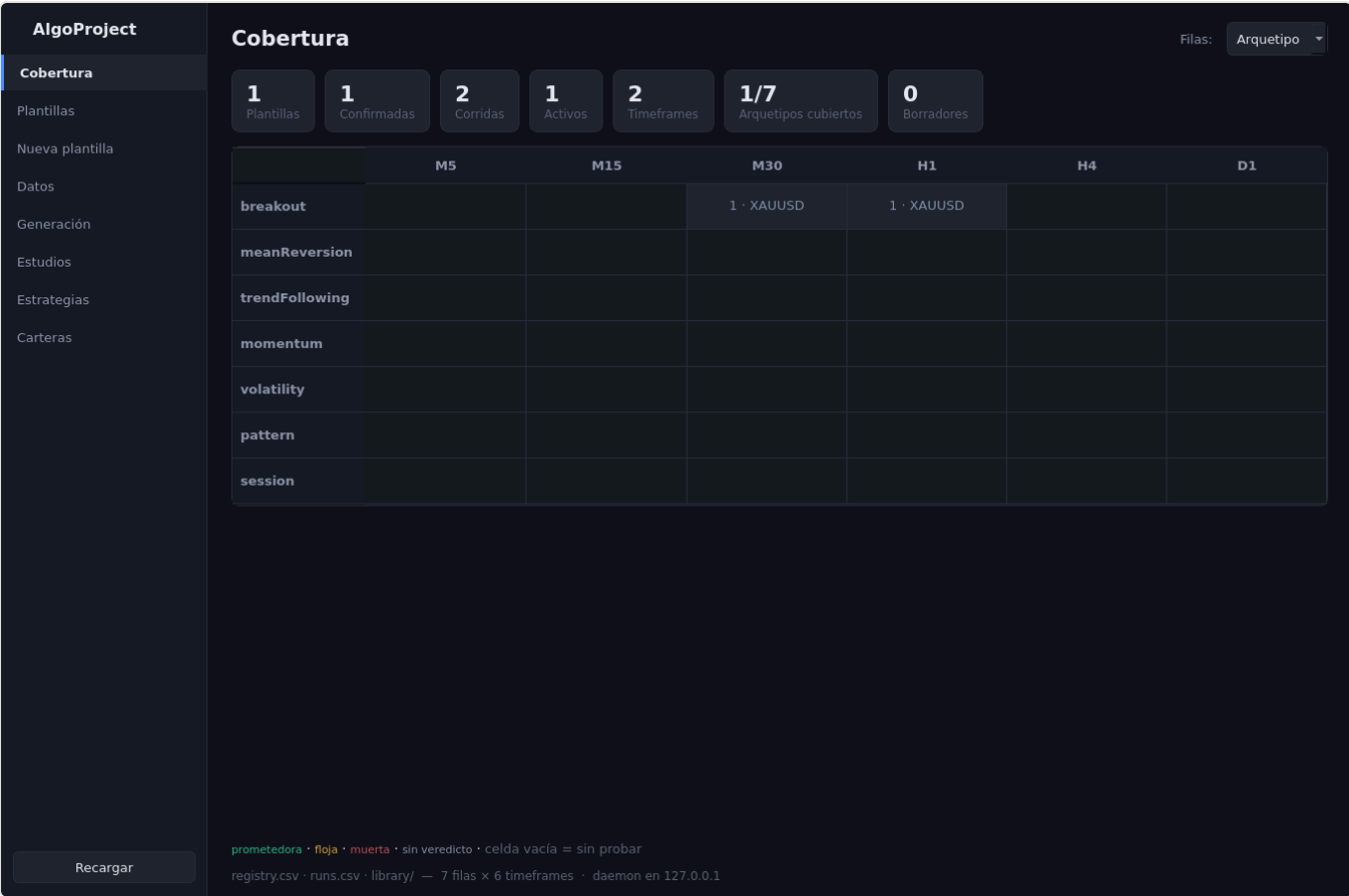
ruta	qué es	cuándo se escribe
<code>AlgoData/templates/registry.csv</code>	la fila de la plantilla, con su <code>status</code> y su <code>notes</code>	al pulsar Guardar en la ficha
<code>AlgoData/templates/runs.csv</code>	la fila de la corrida, con su <code>verdict</code>	al cambiar el desplegable de veredicto
<code>AlgoData/templates/drafts/<nombre>.json</code>	el borrador del brief que redacta el chat	al terminar la entrevista
<code>sqx/blocks/palettes/<familia>.yaml</code>	la paleta de bloques de un arquetipo	al pulsar Guardar paleta

Los dos CSV los escribe `sqx/templates/registry.py`, **la misma función que usa la línea de comandos**. La ventana no tiene una segunda forma de escribir un CSV: dos escritores para un fichero es como se descuadra una columna.

⚠ **La ventana escribe paletas, nunca la taxonomía.** `sqx/blocks/taxonomy.yaml` lleva las etiquetas y lo rellena otra persona (`docs/encargos/6-taxonomia-bloques.md`); desde aquí solo se escriben los `overrides` de una paleta. Son dos ficheros distintos a propósito.

Cómo se lee el resultado

Cobertura — la primera pantalla



Arriba, siete contadores. **Todos explican qué cuentan al pasar el ratón por encima**, porque un número que nadie sabe definir es un número con el que nadie debería decidir. El que más dice es *Arquetipos cubiertos*: **1/7** significa que de los siete arquetipos que el catálogo conoce, has tocado uno. Esa distancia **es** la respuesta a «de qué tengo poco».

Debajo, la rejilla. Las columnas son siempre los timeframes —es el eje con vocabulario corto, fijo y ordenado—. Las filas las eliges arriba a la derecha: **Arquetipo**, **Activo** o **Plantilla**.

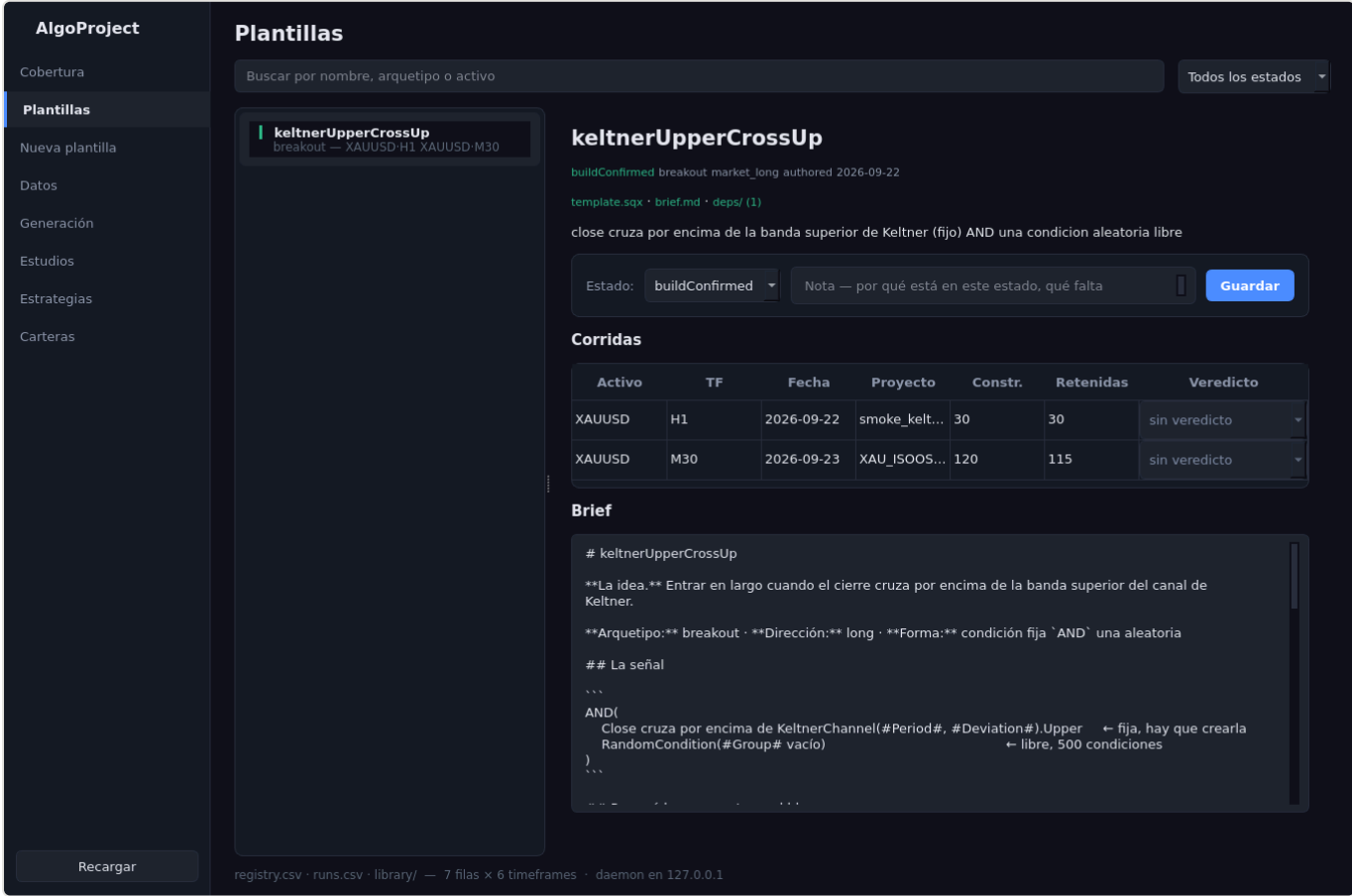
Cada celda dice cuántas corridas hay y en qué mercados, y **su color es el mejor veredicto que hay dentro**, no un promedio. Es deliberado: una celda es una invitación a abrirla, y promediar haría que una celda con una corrida prometedora y dos muertas argumentase en contra de mirarla.

color	significa
verde	alguna corrida quedó como promising
ámbar	la mejor quedó como weak
rojo	todas dead
gris	probada, sin veredicto escrito todavía
celda vacía	sin probar . Es el hueco que la matriz existe para enseñar

Las filas de arquetipo que nadie ha tocado **siguen apareciendo**, vacías. Si se generaran a partir de los datos desaparecerían, y con ellas justo la información que buscas.

Un clic en una celda con datos abre esa plantilla.

Plantillas — la ficha

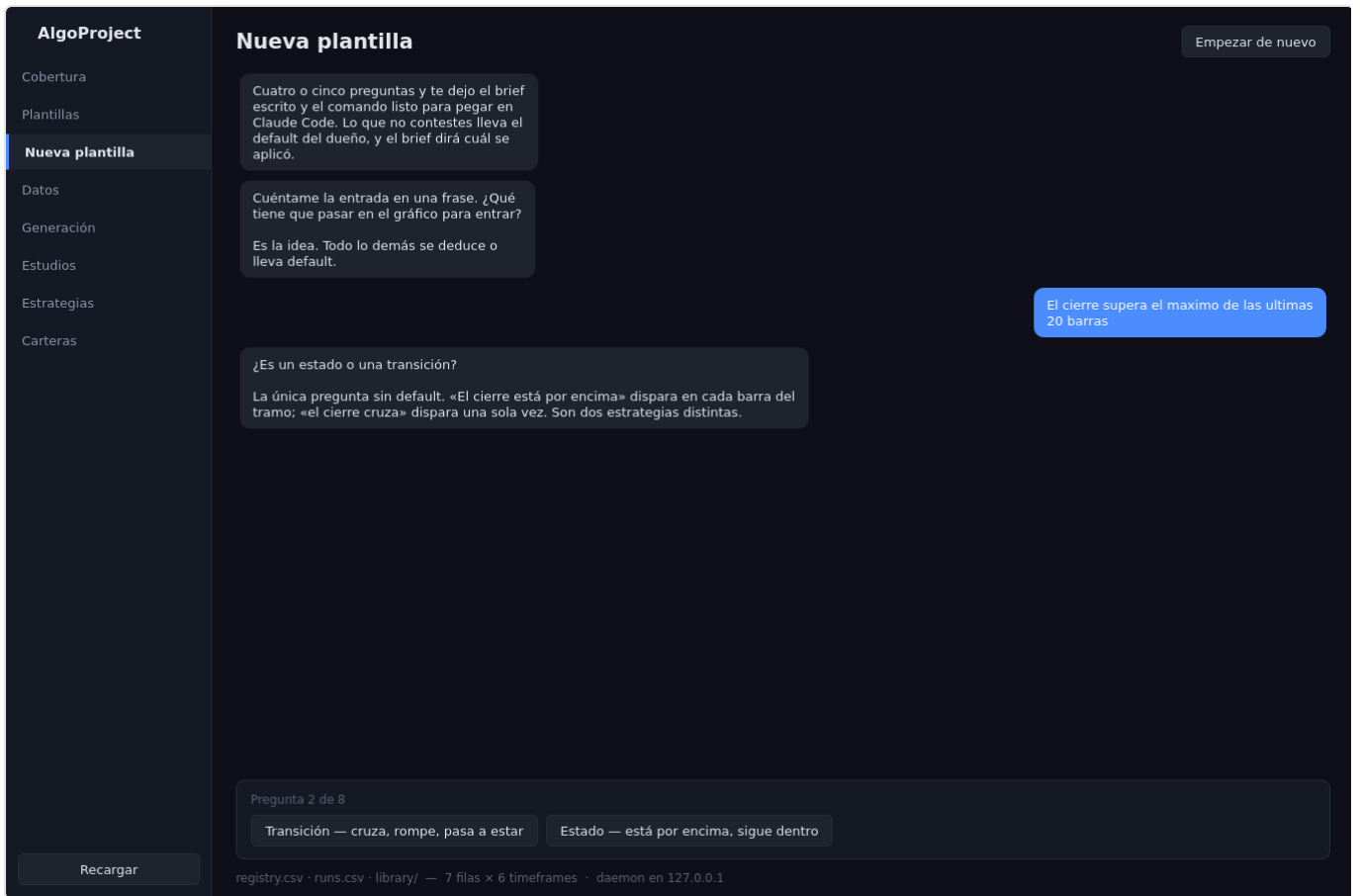


A la izquierda la lista, con buscador (nombre, arquetipo o activo) y filtro por estado. Los borradores del chat aparecen marcados como tales: **no son plantillas**, no tienen `.sqx` , y por eso no cuentan en la matriz.

A la derecha la ficha. En orden:

- **Las etiquetas:** estado, arquetipo, forma, origen y fecha. El estado explica su significado al pasar el ratón: `draft` es XML sin comprobar, `validated` es XML que resuelve, `buildConfirmed` es el único que prueba algo — una construcción real sacó estrategias y llevan de verdad su bloque.
- **La línea de disco:** `template.sqx` , `brief.md` , `deps/` . En verde lo que está, en rojo lo que falta. El registro es una **afirmación**; esta línea es la **prueba**. Importa porque SQX no da error con una plantilla cuyos bloques no están instalados: la quita del builder en silencio.
- **Estado y nota**, con su botón Guardar.
- **Corridas:** una fila por mercado y timeframe, con las estrategias construidas y retenidas, y el veredicto como desplegable. El veredicto se escribe **al cambiarlo**, sin botón: es un juicio tuyo, no un formulario.
- **Brief:** el texto en castellano tal cual lo escribió quien la autoró. La ventana no lo resume y no lo deja editar por detrás del fichero.

Nueva plantilla — el chat



Ocho preguntas como mucho. Las que llevan default del dueño traen un botón **Elige tú** que lo aplica y lo deja escrito en el brief; las que no lo llevan, no se pueden saltar.

Una pregunta no tiene default y nunca lo tendrá: estado contra transición. «El cierre *está* por encima de la banda» dispara en cada barra del tramo; «el cierre *cruza* la banda» dispara una sola vez. Son dos estrategias distintas, no dos maneras de decir lo mismo, y adivinarla gasta CPU para construir lo que no pediste.

Al terminar:

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Datos

Generación

Estudios

Estrategias

Carteras

Nueva plantilla

Empezar de nuevo

elige tu

high20BreakStop

Nombre de la plantilla, en camelCase. Sin símbolo y sin timeframe.

Una plantilla no pertenece a ningún mercado: eso es una fila de runs.csv.

Lista. Brief escrito en /home/sergioguslw/Desktop/AlgoData/templates/drafts/high20BreakStop.json.

Defaults aplicados: direction, random, exit.

Pega esto en Claude Code

```
/strategy-template high20BreakStop
```

Idea: El cierre supera el maximo de las ultimas 20 barras
Lógica: transición (cruza / rompe / pasa a estar) — ya preguntado, no lo vuelvas a preguntar.
Dirección: long
Arquetipo: breakout
Aleatorias: una condición aleatoria libre (#Group# vacío, 500 condiciones nativas)
Orden de entrada: stop
Salidas: la pila del esqueleto sin tocar

Y esto es lo que la skill ejecutará:

- ¿Existe ya la condición, y qué grupos la agrupan?
python3 -m sqx.inspect.vocabulary <termino>
- Instala el bloque en las dos instalaciones (si hay que autorarlo)
python3 -m sqx.blocks.install /home/sergioguslw/Desktop/AlgoData/templates/library/high20BreakStop/deps/blocks.xml --role conductor
- Comprueba que la instalación que construye no se queda sin el bloque
python3 -m sqx.inspect.vocabulary --diff custodian
- Emite la plantilla
python3 -m sqx.templates.build high20BreakStop /home/sergioguslw/Desktop/AlgoData/templates/library/high20BreakStop/deps/blocks.xml <CBlock_key> /home/sergioguslw/Desktop/AlgoData/templates/library/high20BreakStop/template.sqx
- Da de alta la plantilla en el registro
python3 -m sqx.templates.registry --set name=high20BreakStop --set archetype=breakout --set status=validated

Copiar el prompt

Copiar el brief en JSON

Recargar

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

Te quedan tres cosas: el **borrador del brief** escrito en disco, el **prompt** para pegar en Claude Code —que ya lleva todas las respuestas, para que la skill no vuelva a preguntar lo que ya contestaste—, y **la lista de comandos que la skill va a ejecutar**, para que sepas lo que viene. La ventana no ejecuta ninguno.

Fíjate en la línea *Defaults aplicados: direction, random, exit*. El brief deja constancia de qué preguntas te saltaste y qué default las rellenó. Un default aplicado en silencio y nunca escrito es como se pierde una decisión.

Paletas — qué puede sortear el builder

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Datos

Generación

Estudios

Estrategias

Carteras

Paletas de bloques

Ruptura · ★ Ruptura — base

Clonar

Borrar

Sin etiquetar → fuera (la paleta E)

Buscar bloque o forma en las 8...

Guardar

condiciones 64 de 588 · indicador 37/171 · nivel 24/85 · 104 elegidos a mano · 663 sin etiquetar

Ruptura default Paleta por defecto de la familia. Sin curar todavía; el criterio de qué bloques la forman lo pone el encargo de autoría, no el generador.

△ 64 condiciones: fuera del 90–170 que pide el dueño. Faltan bloques por elegir.

Una paleta gobierna los huecos LIBRES de una plantilla. Un hueco atado a un grupo sortea ese grupo y la ignora; un bloque fijo es parte del esqueleto. Medido el 2026-09-24. 66 apagados están en un grupo — esos se escapan por esa vía.

ADX 0/8

ATR 3/8

ATR Percent 2/4

ATR Percent Rank 3/6

Anchored VWAP 0/6

Aroon 0/6

Avg Volume 2/4

Awesome Oscillator 0/8

BH Ergodic 0/6

Bar And Time 3/25

Bar Range 0/1

Bears Power 0/8

Bollinger Bands 6/16

BookTriggers_user 4/26

BreakoutLong 5/7

BreakoutShort 4/5

Bulls Power 0/8

Bloque	Forma escrita	Papeles	Grupos	Etiqueta	Peso
CBlock_AroonDownCross	Aroon Down crosses above ...	señal	—	— sin etiquetar	— taxonomía
CBlock_AroonUpCross	Aroon Up crosses above Aro...	señal	BookTriggers	— sin etiquetar	— taxonomía
CBlock_BBBreakoutDown	Close crosses below lower ...	señal	—	— sin etiquetar	3 característico
CBlock_BBBreakoutUp	Close crosses above upper ...	señal	BookTriggers	— sin etiquetar	3 característico
CBlock_DidiCrossDown	Didi fast line crosses below ...	señal	—	— sin etiquetar	— taxonomía
CBlock_DidiCrossUp	Didi fast line crosses above ...	señal	BookTriggers	— sin etiquetar	— taxonomía
CBlock_NormMACDBuy	MACD below zero and crosse...	señal	BookTriggers	— sin etiquetar	— taxonomía
CBlock_NormMACDSell	MACD above zero and crosse...	señal	—	— sin etiquetar	— taxonomía
CBlock_PMaxFlipDown	Close crosses below ATR ...	señal	—	— sin etiquetar	2 encaja
CBlock_PMaxFlipUp	Close crosses above ATR ...	señal	BookTriggers	— sin etiquetar	2 encaja
CBlock_QQECrossDown	QQE fast crosses below QQE ...	señal	—	— sin etiquetar	— taxonomía
CBlock_QQECrossUp	QQE fast crosses above QQE...	señal	BookTriggers	— sin etiquetar	— taxonomía
CBlock_QQEModBuy	QQE value crosses above ...	señal	BookTriggers	— sin etiquetar	— taxonomía
CBlock_QQEModSell	QQE value crosses below ...	señal	—	— sin etiquetar	— taxonomía
CBlock_RMICrossDown	TDI RSI crosses below its ...	señal	—	— sin etiquetar	— taxonomía
CBlock_RMICrossUp	TDI RSI crosses above its ...	señal	BookTriggers	— sin etiquetar	— taxonomía
CBlock_RSIVWMAEntryLong	Close above SMA200 and RS...	señal	BookTriggers	— sin etiquetar	— taxonomía
CBlock_RSIVWMAEntryShort	Close below SMA200 and RSI...	señal	—	— sin etiquetar	— taxonomía
CBlock_SMICrossDown	DSS crosses below its trigger...	señal	—	— sin etiquetar	— taxonomía

registry.csv · runs.csv · library/

— 7 filas × 6 timeframes · daemon en 127.0.0.1

Esta es la vista que estrecha el vocabulario del builder, y es una **librería**: no hay tres paletas fijas, hay las que haya. Vienen tres por defecto, una por familia, marcadas con ★, y encima de ellas se clona todo lo demás.

Arriba, de izquierda a derecha:

- **El selector de paleta**, agrupado por familia: `Ruptura · ★ Ruptura — base`. Las ★ son las de por defecto.
- **Clonar**: copia la paleta abierta con el nombre que le des y la abre. Es el camino normal — parte de una por defecto y personalízala.
- **Borrar**: solo para las tuyas. En una por defecto está deshabilitado y dice por qué.
- **Qué hacer con lo no etiquetado**: `entra a peso 1` (la paleta hereda la taxonomía y tú solo corriges) o `fuera` (**la paleta ES su lista**: solo entra lo que hayas elegido). La segunda es la que hace útil una paleta curada a mano sin esperar a que la taxonomía esté etiquetada.
- **El buscador**, que ignora la categoría: encuentra un bloque sin saber en cuál de las 83 lo archivó SQX.
- **Los recuentos**, en su propia línea. El primero, **condiciones**, va en verde o en rojo según caiga dentro del **90–170 que pide el dueño** por build: por debajo el builder no tiene de dónde combinar, por encima vuelve el sobreajuste. Si una paleta se sale, la ficha de debajo lo dice y además apunta hacia dónde — faltan bloques o sobran.

Debajo, la ficha de la paleta abierta: su familia, si es por defecto o tuya, de cuál es clon, y su nota. Y si no ha elegido nada todavía, un aviso en ámbar diciéndolo con todas las letras — porque una paleta vacía y una paleta que deja pasar todo se ven igual, y no son lo mismo.

A la izquierda, las categorías con cuántos bloques tienen encendidos. A la derecha, los bloques: su clave, **su forma escrita** —que es lo único que dice de verdad qué comprueba—, sus papeles, los grupos que lo agrupan, su etiqueta y el peso.

La lista de bloques es siempre la misma — son los 767 del vocabulario del builder — y eso no cambia entre familias. Lo que cambia por familia son las dos últimas columnas y los recuentos.

- **Etiqueta:** lo que `taxonomy.yaml` dice de ese bloque **para la familia que tienes elegida**. Es la columna que hace visible el cambio de familia. `- sin etiquetar` significa que nadie ha dicho nada todavía, y entonces las tres familias lo tratan igual.
- **Peso:** lo único editable, y es un *override*. `- taxonomía` significa «haz lo que diga la etiqueta»; cualquier otro valor la pisa **solo para esta paleta**. `0` apaga el bloque y lo pone en gris, `3` lo prefiere. Se guarda con el botón; hasta entonces el recuento dice cuántos llevas sin guardar.

Un bloque apagado se ve en gris en toda su fila, y el contador de su categoría baja: `ADX 8/8` pasa a `ADX 5/8`. Ésa es la forma rápida de comprobar que una paleta está estrechando de verdad.

△ La línea gris de debajo repite el límite, y hay que tomárselo en serio: **una paleta gobierna los huecos LIBRES de una plantilla**. Un hueco atado a un grupo sorteja ese grupo y la ignora, y un bloque fijo es parte del esqueleto. Está medido el 2026-09-24 y está en `knowhow/authoring/builder-block-switches.md`. Cuando un bloque apagado está en un grupo, su celda de **Grupos** se pone en ámbar y lo dice.

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Datos

Generación

Estudios

Estrategias

Carteras

Recargar

Plantillas

Buscar por nombre, arquetipo o activo

Todos los estados

keltnerUpperCrossUp

breakout — XAUUSD·H1 XAUUSD·M30

keltnerUpperCrossUp

buildConfirmed breakout market_long authored 2026-09-22

template.sqx · brief.md · deps/ (1)

close cruza por encima de la banda superior de Keltner (fijo) AND una condicion aleatoria libre

Estado: buildConfirmed Nota — por qué está en este estado, qué falta Guardar

Qué rellena el builder

1 hueco(s) libre(s): la paleta sí los gobierna.
RandomCondition2 (condition) libre — la paleta lo gobierna
Fijos en la plantilla, fuera de toda paleta: CBlock_CloseCrossesAboveKCUpper, MarketPositionIsLong

Corridas

Activo	TF	Fecha	Proyecto	Constr.	Retenidas	Veredicto
XAUUSD	H1	2026-09-22	smoke_kelt...	30	30	sin veredicto
XAUUSD	M30	2026-09-23	XAU_ISO0...	120	115	sin veredicto

Brief

keltnerUpperCrossUp

La idea. Entrar en largo cuando el cierre cruza por encima de la banda superior del canal de Keltner.

Arquetipo: breakout · **Dirección:** long · **Forma:** condición fija `AND` una aleatoria

La señal

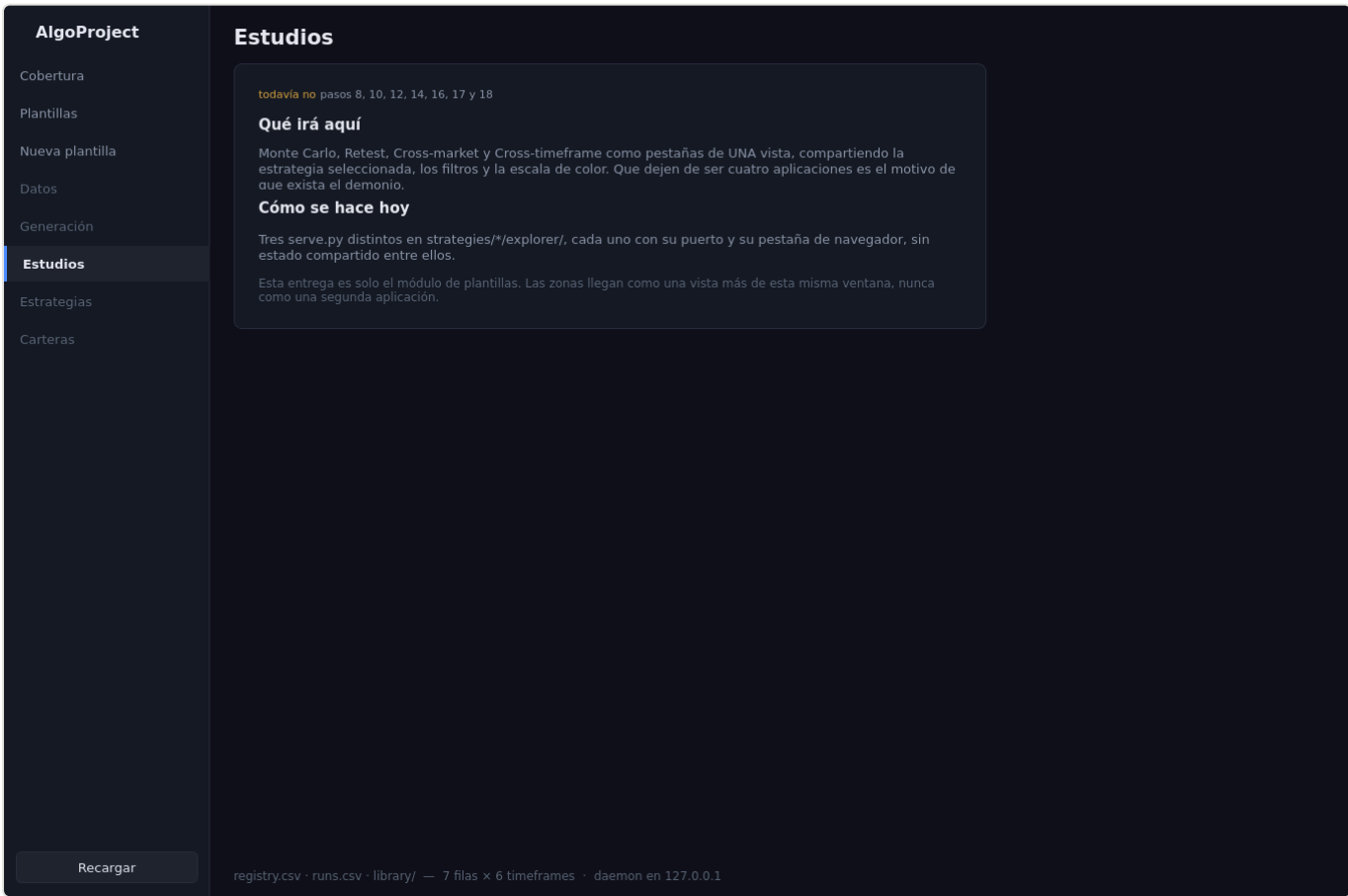
...

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

Por eso la ficha de cada plantilla lleva ahora **Qué rellena el builder**: un hueco por línea, diciendo si es libre —y entonces la paleta lo gobierna— o a qué grupo está atado, y debajo los bloques fijos, que están fuera de toda paleta. Si esa línea dice que no hay huecos libres, la vista de paletas no puede hacer nada por esa

plantilla y hay que tocar el grupo.

Las otras cinco zonas



Datos y Carteras se pueden abrir, y no hacen nada todavía — cada una dice qué irá ahí, cómo se hace ese trabajo hoy mientras tanto, y qué pasos del workflow cubre. Generación, Estrategias y Puerta IS/OOS ya están construidas y tienen su propia página: `02-la-ventana.pdf > 48-app-generacion` , `02-la-ventana.pdf > 44-app-estrategias` y `02-la-ventana.pdf > 45-app-puerta` . Las zonas pendientes están en la barra lateral desde el primer día a propósito: este módulo es la primera rebanada de un producto, y las zonas que faltan llegarán como una vista más de esta misma ventana, nunca como una segunda aplicación.

Se pulsan en vez de estar apagadas por un motivo concreto: en Qt un botón deshabilitado no recibe el ratón, así que **su tooltip no se muestra nunca**. Cinco entradas grises con una explicación que nadie puede leer no explican nada.

Un ejemplo completo

Idea: *«el cierre supera el máximo de las últimas 20 barras»*.

1. `bin/algoui` . Abre en Cobertura. `1/7` arquetipos, dos corridas, un solo activo.
2. **Nueva plantilla**. Escribes la idea. Contestas **Transición** — quieres que dispare al romper, no en cada barra por encima. Arquetipo **breakout**. Orden de entrada **Stop**. Dirección, aleatorias y salidas: **Elige tú** en las tres. Nombre: `high20BreakStop` .
3. La ventana escribe `~/Desktop/AlgoData/templates/drafts/high20BreakStop.json` y te da el prompt.

4. **Copiar el prompt** y pegarlo en Claude Code. Ahí sí: `/sqx-strategy-template` mira el vocabulario de la instalación, autora el bloque si no existe, emite el `.sqx` y da de alta la fila.
5. Vuelves a la ventana, **Recargar**. La plantilla ya está en la lista, el borrador ha desaparecido, y la fila `breakout` de la matriz sigue sin corrida nueva — porque autorar no es construir. Eso es `/template-run`.
6. Cuando esa corrida termine y la hayas mirado, abres su ficha y le pones el veredicto.

Qué NO te dice

- **No te dice si una plantilla es buena.** `buildConfirmed` significa que SQX la acepta y que las estrategias llevan su bloque. No dice que gane dinero. Eso lo deciden los pasos 12 a 19 del workflow, no esta ventana.
- **El veredicto de una corrida es tuyo, no suyo.** La ventana lo guarda y lo pinta; no lo calcula. Una celda verde significa «tú dijiste que era prometedora», nada más.
- **La matriz cuenta corridas, no calidad ni tamaño.** Una celda con `1` puede ser una construcción de humo de 30 estrategias y 29 segundos. El número de estrategias está en la ficha, no en la celda.
- **No sabe lo que hay dentro de un `.sqx`.** La línea de disco dice si el fichero existe, no si su contenido es el que el registro afirma. Para eso está `sqx/inspect/template_check.py`. La excepción es el panel de huecos, que sí lo abre y lee sus bloques aleatorios.
- **Una paleta guardada todavía no llega a SQX.** La ventana escribe el YAML; quien lo tiene que aplicar al `<BuildingBlocks>` de la tarea Build es el builder, y esa pieza no está escrita. Hasta entonces una paleta es una decisión anotada, no un efecto.
- **La paleta no etiqueta.** Los pesos salen de `taxonomy.yaml`, que a día de hoy está entero sin etiquetar: `767 sin etiquetar`. Mientras siga así, las tres paletas son idénticas y no estrechan nada.

Si algo falla

- **el demonio no respondió en el puerto 8765** — hay otra cosa ocupando el puerto, o el demonio se cayó al arrancar. Arráncalo a mano con `python3 -m ui.daemon.serve` y lee el error; si el puerto está ocupado, cámbialo en `ui_port` de `config/machine.yaml`.
- **La ventana abre vacía y los contadores están a cero** — `AlgoData/templates/` no existe o no tiene CSV. No es un fallo: es una librería sin nada dentro.
- **Una plantilla aparece en rojo en la línea de disco** — su fila está en el registro pero su carpeta no está completa. Suele pasar al traer un registro de otra máquina sin traer `library/`.
- **qt.qpa.plugin: could not load the Qt platform plugin "xcb"** — **pasó en esta máquina**, el 2026-09-24, la primera vez que se abrió la ventana de verdad. El plugin `xcb` de Qt necesita cuatro librerías que Ubuntu no trae puestas: `libxkbcommon-x11.so.0`, `libxcb-icccm.so.4`, `libxcb-keysyms.so.1` y `libxcb-xkb.so.1`. El mensaje que da Qt culpa a `libxcb-cursor0`, que **sí estaba** — o sea que el mensaje miente sobre cuál falta. Quién falta de verdad lo dice esto:

```
bash ldd ~/.local/lib/python3.10/site-packages/PySide6/Qt/plugins/platforms/libqxcb.so | grep "not found"
```

Como `sudo` pide contraseña aquí, no se instalan con `apt install`: se bajan y se extraen en el home, que es lo que ya está hecho en `~/.local/lib/qt-xcb/`.

```
bash apt-get download libxkbcommon-x11-0 libxcb-icccm4 libxcb-keysyms1 libxcb-xkb1 for d in *.deb; do dpkg -x "$d" x; done mkdir -p ~/.local/lib/qt-xcb && cp -P x/usr/lib/x86_64-linux-gnu/*.so* ~/.local/lib/qt-xcb/
```

bin/algoui añade esa carpeta al LD_LIBRARY_PATH él solo, así que una vez hecho esto no hay que acordarse de nada. En un ordenador nuevo con sudo a mano, sudo apt install libxkbcommon-x11-0 libxcb-icccm4 libxcb-keysyms1 libxcb-xkb1 hace lo mismo y la carpeta del home sobra.

38. La aplicación de escritorio — activos

La segunda zona de la ventana. Contiene **todo lo que hay en `assets/`** : los diecinueve instrumentos y los cuatro ficheros que deciden para todos a la vez, con sus costes, sus ventanas temporales, sus rangos de MC Retest y su universo de retest — y se pueden cambiar desde ahí.

Qué pregunta responde

Antes había que abrir veintitrés ficheros YAML para contestar a esto. Ahora son cuatro clics:

- **¿Qué activos tengo y cuáles están bloqueados?** Un activo con un coste obligatorio sin pactar no deja escribir ni una plantilla ni un proyecto (regla dura 5). La lista lo dice en rojo, y la ficha dice exactamente qué campo falta y qué pasa por eso.
- **¿Qué le estoy cobrando de verdad a este instrumento, y en qué unidad?** El spread, la comisión, el slippage y los dos swaps, cada uno con su unidad, al lado de lo que SQX lleva hoy — que casi nunca está en la misma unidad — y con la línea que dice por qué difieren.
- **¿Entre qué fechas se construye y se testea?** Los tres tramos, con el aviso cuando uno está RESERVADO y gastarlo es una puerta de una sola dirección.
- **¿Qué forma pueden tener las estrategias que genero?** La doctrina de construcción entera, con el comentario del propio fichero al lado de cada valor.

Cuándo lo usas, y cuándo no

Lo usas cuando el bróker te da una cifra de verdad y hay que sustituir una provisional; cuando das de alta un instrumento nuevo; cuando ajustas una ventana; cuando quieres ver de un vistazo qué te falta por decidir; y cuando alguien —tú o una sesión— va a construir sobre un activo y hay que mirar antes qué arrastra.

No lo usas para nada que hable con SQX. La ventana **no abre el maestro, no arranca un worker y no lee un instrumento del install**. `instrument:` y `sqx_now:` son hechos leídos de SQX, y quien los refresca sigue siendo `python3 -m sqx.inspect.instruments` ; la ventana solo los enseña y te deja escribirlos a mano si los tienes delante.

Tampoco sustituye a la preflight. Antes de autorizar nada sigue siendo obligatorio `python3 -m core.assets <SÍMBOLO>` : la ventana enseña lo mismo, pero el código de salida —2 bloqueado, 3 esquema roto— es lo que para el trabajo, y eso solo lo da el comando.

Antes de empezar

- `pip install -r requirements.txt` . La zona añade **una** dependencia: `ruamel.yaml` , el YAML que conserva los comentarios. Sin ella la app no arranca.
- Nada más. No hace falta SQX, ni un worker, ni datos exportados.

Cómo se ejecuta

```
algoui
```

La misma ventana de siempre (capítulo 35). En la barra de la izquierda, **Activos**.

Qué produce

Escribe, y solo escribe, dentro de `assets/` :

lo que tocas	dónde acaba
un coste, la sesión, el feed, los hechos del instrumento, los rangos MC Retest	<code>assets/symbols/<SÍMBOLO>.yaml</code>
las fechas de un tramo	<code>assets/_policy.yaml</code> , bajo <code>segments: <SÍMBOLO>:</code>
una categoría del universo de retest	<code>assets/_markets.yaml</code>
la doctrina de construcción	<code>assets/_build.yaml</code>
el esquema de las dos clases de coste	<code>assets/_classes.yaml</code>
dar de alta un activo	un <code>.yaml</code> nuevo en <code>symbols/</code> y su bloque en <code>_policy.yaml</code>
retirar un activo	se mueve a <code>assets/symbols/_retired/<SÍMBOLO>.yaml</code>

Después de cada escritura regenera `assets/INDEX.md` , así que el índice nunca contradice a los ficheros.

Los comentarios se conservan. Cambiar un valor cambia esa línea y nada más: las cabeceras, los Δ , las unidades y los párrafos que explican por qué el porcentaje del swap es ANUAL siguen donde estaban.
Verificado el 2026-09-24: leer y reescribir los cinco ficheros sin tocar nada devuelve un resultado idéntico byte a byte.

Cómo se lee el resultado

La lista y la ficha

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Datos

Generación

Estudios

Estrategias

Carteras

19 activos · 8 bloqueados · 11 sólo con cifras provisionales · 0 retirados

Fichero enteroNuevo activoRetirar

ACTIVOS

AUDJPY

AUDUSD

BRENT

CADJPY

DAX40

DJ30

EURJPY

EURUSD

GBPJPY

GBPUSD

NIKKEI225

SP500It

USA500

USATEC

USDCAD

USDCHE

USDJPY

XAGUSD

XAUUSD

COMPARTIDO — decide para todos

Doctrina de construcción

Clases de coste

no_forex

Infinox

XAUUSD_DukasM1

Infinox sesión

XAUUSD_ftmo

datos 2003-05-05 → 2026-09-22

COSTES PROVISIONALES

spread_is, spread_oos, commission, slippage_is, slippage_oos, swap_long, swap_short

— Hay cifra y no bloquea, pero no está pactada con el bróker: TODO resultado con coste producido con ella arrastra esa advertencia.

RANGOS MC RETEST SIN DECIDIR

min_distance

— No bloquea: sólo vuelve ininterpretable esa tarea del MC Retest.

Costes — doble clic en una fila para cambiarla

campo	usar	unidad	SQX hoy	por qué
spread_is	5.0	points	5	PROVISIONAL 2026-09-21 — default de SQX, no una cifra pactada con...
spread_oos	10.0	points	10	PROVISIONAL 2026-09-21 — default de SQX, no una cifra pactada con...
commission	0.001	pct_of_notional	{'method': 'SizeBased', 'value': 8}	PROVISIONAL 2026-09-22 — conversion de unidad del default de SQX...
slippage_is	2.5	points	0.0	PROVISIONAL 2026-09-23 — la mitad del spread de su tramo, ...
slippage_oos	5	points	0.0	PROVISIONAL 2026-09-23 — la mitad del spread de su tramo, ...
swap_long	-7	pct_annual	{'type': 'points', 'value': -73.42}	PROVISIONAL 2026-09-22 — cifra de trabajo puesta por el dueño, no ...
swap_short	-7	pct_annual	{'type': 'points', 'value': 38.76}	PROVISIONAL 2026-09-22 — cifra de trabajo puesta por el dueño, no ...

assets/symbols/XAUUSD.yaml · sus tramos viven en assets/policy.yaml · proyectos del maestro que lo usan: AUDJPY, Builder, EURUSD, Retester, USDJPY, XAUUSD, XAUUSD_Breakout_H1

Tramos — doble clic en una fecha para cambiarla

tramo	desde	hasta	spread	para qué
build	2008	2017	is	Construcción. Es la única muestra que el generador ve.
oos1	2018	2022	oos	Todo lo que va desde el retest hasta los SPP: decaimiento, Monte Carlo, MC Retest, cross-market, SPP.
oos2	2023	2026-08-30	oos	RESERVADO para Walk Forward Correlation y Walk Forward Matrix, que son las últimas pruebas. No ...

MC Retest — lo que sortea cada método, en puntos y absoluto

rango	min	max	SQX hoy
spread	5	30	[{'min': 1, 'max': 5}, {'min': 5, 'max': 12}]
slippage	5	20	[{'min': 0, 'max': 5}, {'min': 0, 'max': 10}]
min_distance	null	null	{'min': 0.0, 'max': 10.0}

Universo de retest — doble clic para editar una categoría

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

A la izquierda, los diecinueve. El color es el estado y el tooltip lo deletrea:

color	qué significa
rojo	bloquea la autoría: falta un coste obligatorio, o el esquema está roto
ámbar	funciona, pero alguna cifra es PROVISIONAL y todo resultado con coste la arrastra
normal	nada que decidir

Debajo del título, las fichas: clase, bróker, feed, sesión, y el rango de datos que SQX tiene. En rojo si la sesión está sin decidir o si SQX no tiene ese feed.

La tira de debajo es **la preflight entera**, en el orden en el que se actúa sobre ella: primero lo que rompe la corrida, después lo que bloquea la autoría, al final lo que solo mancha el resultado.

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Datos

Generación

Estudios

Estrategias

Carteras

19 activos · 8 bloqueados · 11 sólo con cifras provisionales · 0 retirados

Fichero enteroNuevo activoRetirar

ACTIVOS

AUDJPY

AUDUSD

BRENT

CADJPY

DAX40

DJ30

EURJPY

EURUSD

GBPJPY

GBPUSD

NIKKEI225

SP500R

USA500

USATEC

USDCAD

USDCHF

USDJPY

XAGUSD

XAUUSD

COMPARTIDO — decide para todos

Doctrina de construcción

Clases de coste

DAX40

no_forex Infinox DAX40_DukasM1_Infinox sesión GER40.cash_fimo datos 2013-09-30 → 2026-01-16

BLOQUEA LA AUTORIA spread_is, spread_oos, commission — La preflight sale con 2 (regla dura 5): sin ese coste pactado no se escribe ninguna plantilla ni proyecto para este activo.

COSTES PROVISIONALES slippage_is, slippage_oos, swap_long, swap_short — Hay cifra y no bloquea, pero no está pactada con el bróker: TODO resultado con coste producido con ella arrastra esa advertencia.

TRAMOS SIN FECHAS build, oos1, oos2 — No bloquea, pero 'window()' se niega a inventar una ventana, así que una tarea sobre ese tramo falla al configurarse.

RANGOS MC RETEST SIN DECIDIR spread, slippage — No bloquea: sólo vuelve ininterpretable esa tarea del MC Retest.

Costes — doble clic en una fila para cambiarla

campo	usar	unidad	SQX hoy
spread_is	SIN DECIDIR	points	100
spread_oos	SIN DECIDIR	points	100
commission	SIN DECIDIR	pct_of_notional	{'method': 'SizeBased', 'value': 1}
slippage_is	50	points	null
slippage_oos	50	points	null
swap_long	5	pct_annual	null
swap_short	5	pct_annual	null

Tramos — doble clic en una fecha para cambiarla

tramo	desde	hasta	spread	para qué
build	null	null	is	Construcción. Es la única muestra que el generador ve.
oos1	null	null	oos	Todo lo que va desde el retest hasta los SPP: decaimiento, Monte Carlo, MC Retest, cross-market, SPP.
oos2	null	null	oos	RESERVADO para Walk Forward Correlation y Walk Forward Matrix, que son las últimas pruebas. No usar an

MC Retest — lo que sortea cada método, en puntos y absoluto

rango	min	max	SQX hoy
spread	null	null	null
slippage	null	null	null

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

El DAX40 lo enseña completo: tres costes sin pactar, los tres tramos sin fechas, los dos rangos de MC Retest sin decidir, y cuatro cifras provisionales que sí tiene.

La tabla de costes lleva SIN DECIDIR en rojo cuando el campo es obligatorio para su clase. La columna SQX hoy está en la unidad de SQX, que no es la de usar — por eso las dos columnas existen por separado, y por eso el por qué de al lado dice en qué se diferencian.

Cambiar un coste

Doble clic en cualquier celda de la fila:

XAUUSD · spread_is

Se aplica en points. SQX lleva hoy `5`, que no siempre está en la misma unidad. Un coste sin decidir (`null`) BLOQUEA la autoría: la preflight sale con 2 y ninguna plantilla ni proyecto se escribe para este activo. Es a propósito — un número inventado parece decidido.

Usar (points)

Por qué

La app propone una línea con la fecha y el valor que sustituye; acábala. No escribe PROVISIONAL por ti: esa palabra es la que lee `assetcheck.provisional` para marcar el activo, y ponerla sería decidir por ti cuán provisional es.

PROVISIONAL 2026-09-21 — default de SQX, no una cifra pactada con Infinox.

Save

Cancel

Dos campos, y los dos se escriben juntos o no se escribe ninguno. Dejar el valor vacío es escribir `null`, o sea devolver el campo a «sin decidir» y bloquear la autoría a propósito.

Al salir del campo del valor, la app **propone** una línea de `por qué` con la fecha y la cifra que sustituye, y te deja acabarla. No escribe `PROVISIONAL` por ti: esa palabra exacta es la que lee `core.assetcheck.provisional` para marcar el activo en ámbar, y ponerla sería decidir por ti cuán provisional es tu propia cifra.

Cambiar una ventana o un rango

Doble clic sobre la fecha, no sobre la fila:

XAUUSD · build · from

Un año entero (2008) o una fecha (2026-08-30), en ambos casos con los dos extremos INCLUIDOS. Vacío = null, sin decidir. Se escribe en assets/_policy.yaml, donde están los 19 juntos.

2008

Save

Cancel

Un año entero o una fecha, siempre con los dos extremos incluidos. El tramo oos2 sale marcado RESERVADO: es el que gastan la Walk Forward Correlation y la Matrix, y mirarlo antes lo quema.

Los cuatro ficheros compartidos

Debajo de los activos, en COMPARTIDO — decide para todos:

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Datos

Generación

Estudios

Estrategias

Carteras

Activos

19 activos · 8 bloqueados · 11 sólo con cifras provisionales · 0 retirados

Fichero entero

Nuevo activo

Retirar

ACTIVOS

AUDJPY

AUDUSD

BRENT

CADJPY

DAX40

DJ30

EURJPY

EURUSD

GBPJPY

GBPUSD

NIKKEI225

SP500ft

USA500

USATEC

USDCAD

USDCHEF

USDJPY

XAGUSD

XAUUSD

COMPARTIDO — decide para todos

Doctrina de construcción

Clases de coste

Doctrina de construcción — assets/_build.yaml

Qué FORMA puede tener una estrategia: complejidad, órdenes, salidas, money management, precisión y cross-checks. Lo que 'doctrine.py' escribe en TODAS las tareas de un proyecto.

campo	valor	qué dice el fichero
complexity / max_entry_conditions	2	
complexity / max_exit_conditions	2	
complexity / lookback_bars	1	Global lookback. 1 = sólo la última barra cerrada. Subirlo es invitar al sobreajuste.
order_types	EnterAtMarket	Tipos de orden Sólo a mercado, salvo que el dueño pida otra cosa explícitamente.
exits / bars / min_hours	2	
exits / bars / max_hours	24	
exits / by_condition	true	
exits / condition_required	false	
exits / stop_loss	false	
exits / profit_target	false	
exits / trailing_stop	false	
exits / move_sl_to_be	false	
money_management / method	ATRRiskBasedSizingFixedRisk	

registry.csv · runs.csv · library/ — 7 filas x 6 timeframes · daemon en 127.0.0.1

Es el fichero entero como tabla: la ruta del valor, el valor, y **el comentario que el propio YAML lleva encima o al lado**. La explicación no está escrita en la app — se lee del fichero, así que no puede contradecirlo.

Los valores cortos se editan en la celda. Las listas y los textos largos abren una caja, una línea por elemento. El mismo botón **Fichero entero** de arriba enseña así el fichero de un activo, con todo lo que la ficha no muestra.

Dar de alta y retirar

Nuevo activo

La clase decide QUÉ CAMPOS de coste tendrá el fichero: ``forex`` Pares de divisas. Hoy: los 10 de the5ers. · ``no_forex`` Todo lo demás: índices CFD y metales. Hoy: DAX40, DJ30, NIKKEI225, SP500ft, USA500, USATEC, XAUUSD.

Nombre	XAUUSD — es también el nombre del fichero
Bróker	
Símbolo SQX	tal y como lo imprime -symbol action=list
Sesión	vacío = sin decidir, y eso bloquea la autoría
Clase	forex
Tamaño de tick	
Valor del punto	
Distancia mínima	

Los tres últimos son hechos del feed, no decisiones: sácalos con ``python3 -m sqx.inspect.instruments``. Todos los costes nacen ``null`` y el activo queda bloqueado hasta que los pactes.

SaveCancel

La clase decide qué campos de coste tendrá el fichero, así que es lo único que no se puede cambiar después sin reescribirlo. Los tres hechos del instrumento se teclean —sácalos de `python3 -m sqx.inspect.instruments` — y **todos los costes nacen `null`**: el activo entra bloqueado, que es justo lo que se quiere. Un fichero con cifras inventadas es peor que ninguno, porque parece decidido.

Retirar mueve el fichero a `symbols/_retired/`. Deja de existir para `core.assets` y para el índice, pero su bloque de tramos se queda en `_policy.yaml`: si lo restauras, vuelve con sus ventanas intactas. Aparece abajo del todo, en RETIRADOS, con el botón para devolverlo.

Un ejemplo completo

The5ers te pasa por fin el spread real del EURUSD: 0,3 puntos en vez del 0,1 de fábrica.

1. `algoui`, zona **Activos**, `EURUSD` en la lista. Sale en ámbar: seis cifras provisionales.

2. Doble clic en la fila `spread` . La caja dice `points` , y que SQX lleva hoy `0.1` .
3. Escribes `0.3` y sales del campo. El `por qué` se rellena solo con `2026-09-24 – cambiado desde la app: 0.1 → 0.3.`
4. Lo acabas: `...cifra pactada con the5ers por correo el 2026-09-24.` — sin la palabra PROVISIONAL, porque ya no lo es. Guardar.
5. La ficha se redibuja: `spread` ya no aparece en la lista de provisionales.
6. En la terminal, la comprobación que manda:

```
$ python3 -m core.assets EURUSD
AVISO: el tramo `oos2` llega a 2026-08-30 y los datos de `EURUSD_DukasM1_the5ers` acaban en
2026-01-16
# EURUSD – clase `forex`, overrides a aplicar

SQX symbol: EURUSD_DukasM1_the5ers   verificado: 2026-09-03

- **spread**: usar `0.3` points (SQX lleva hoy `0.1`) – 2026-09-24 – cambiado desde la app: 0.1
→ 0.3. cifra pactada con the5ers por correo el 2026-09-24.
...
```

(El AVISO de arriba es de este activo hoy y no tiene que ver con el cambio: la ventana `oos2` llega más lejos que los datos sincronizados, que es el estado normal del tramo más nuevo.)

El `git diff` de ese cambio son **dos líneas**: el valor y su `por qué` .

Qué NO te dice

- **No dice si una cifra es correcta**, solo si está decidida. `use: 0.0` en la comisión es una decisión perfectamente válida para la preflight y a la vez un modelo de coste que infravalora el real diez veces. Eso lo dice el `por qué` , que es texto y no lo valida nadie.
- **No sabe qué lleva SQX ahora mismo**. `sqx_now` es una foto del maestro del día en que alguien corrió `sqx.inspect.instruments` . Si has tocado un proyecto en la GUI desde entonces, la columna miente y la ventana no puede saberlo.
- **No comprueba que la sesión exista** en el proyecto donde se va a usar. Una tarea que nombra una sesión que su proyecto no lleva carga sin quejarse y opera otro horario; de eso se ocupa `sqx/projects/doctrine.py` , no la ventana.
- **No propaga nada**. Cambiar un coste aquí no reconfigura ningún proyecto de SQX ya escrito: los costes viven dentro de cada tarea, y volver a escribirlos es trabajo del módulo que la configura.

Si algo falla

- **ModuleNotFoundError: ruamel** — falta la dependencia nueva: `python3 -m pip install --user ruamel.yaml==0.18.16` .
- **La ventana se queda con la lista vieja después de un cambio** — el botón **Recargar** de abajo a la izquierda relee todo desde el demonio. Pasa si has editado un YAML a mano en paralelo.
- **KeyError al abrir un activo recién copiado a mano** en `symbols/` — le falta su bloque en `_policy.yaml` . Los que se dan de alta desde la ventana lo llevan; a uno copiado a mano hay que ponérselo, aunque sea con las fechas a `null` .

- Un **- ítem** de una lista larga se ha quedado en una sola línea después de guardar. Es cosmético y solo le pasa a los textos que estaban partidos a mano en varias líneas: el contenido es idéntico. Los comentarios no se tocan.

48. La aplicación de escritorio — generación, o por dónde va el proyecto

La zona que responde a «¿por dónde va lo que está corriendo?» sin abrir SQX ni molestarlo. Elige un install y un proyecto y enseña, cada tres segundos, qué tarea está en marcha, qué porcentaje lleva, cuántas estrategias ha procesado de cuántas, cuánto tarda cada una, cuánto lleva la tarea y cuánto lleva el workflow del día. **No arranca, no para, no reconfigura.** Lee ficheros del install y, solo mientras corre en un worker, le pide su línea de estado: el único comando que la regla del dueño permite al custodio a mitad de trabajo, porque no sincroniza nada.

Qué pregunta responde

- **¿Está corriendo, y qué?** La línea de estado: `CORRIENDO · tarea MCR 6 Exits · 4 / 20 estrategias · 13,0 s por estrategia · la tarea lleva 1 min 41 s · workflow de hoy: 23 min`. O `terminado`, otro proyecto corre (el install está con otro) o `parado`. Al final, hace cuántos segundos creció el log: si ese número sube y sube, SQX no escribe.
- **¿Qué tareas tiene el proyecto y en qué estado?** La tabla, en el orden en que SQX las corre: tipo, si está activa en este `start` (lo que decide `stage`, manual `04-sqx-plantillas-y-proyectos.pdf` > `47-proyecto-workflow`), su estado, **hechas / total**, el tiempo que tardó o que lleva, el tiempo por estrategia, su databank de salida y cuántos ficheros hay ya en él.
- **¿Qué está haciendo exactamente ahora?** Las últimas líneas de progreso del log, tal cual las escribe SQX: «Loading backtest data...», «All backtest data prepared», «Task finished in 19 s».

Cuándo lo usas, y cuándo no

Lo usas mientras corre un paso largo en el custodio, para no abrir SQX y no mandarle nada; y al volver a una sesión, para ver qué acabó y qué no.

De dónde salen los números. El *total* de una tarea es lo que había en su databank de entrada cuando empezó, escrito por SQX en el log propio del proyecto (`user/projects/<P>/log/`); un build no tiene total. Las *hechas* mientras corre son «Strategies generated» del estado del worker; al acabar, «Total tested» del mismo log. El tiempo por estrategia es el que SQX declara; si mientras corre dice cero, lo que lleva la tarea entre las hechas. El workflow del día va desde el primer `TASK STARTED` de hoy hasta el último `TASK FINISHED`, o hasta ahora.

No lo usas para lanzar ni parar: no hay botón, a propósito. Arrancar y parar sigue siendo `bin/sqx-worker.sh` y las skills, con las reglas de carriles de `sqx/CLAUDE.md`. Tampoco es un recuento exacto del databank en curso: SQX escribe los `.sqx` al sincronizar, así que la salida de la tarea que corre va con retraso hasta que guarda.

Antes de empezar

Nada. Vale con el maestro abierto (a él no se le pide nada nunca) y con el custodio en mitad de un trabajo (solo `status`). Los installs son los de `config/machine.yaml`: conductor, custodio y maestro.

Cómo se ejecuta

```
bin/algoui --zone "Generación"
```

O desde la barra lateral. Elegir el install y el proyecto; la zona se refresca sola cada tres segundos mientras está a la vista y deja de leer cuando cambias de zona.

control	qué hace
install	custodian (SQX_w2), conductor (SQX_w1) o master . El maestro solo se lee
proyecto	cualquier carpeta de user/projects de ese install con project.cfx

Qué produce

Nada. No escribe ni un fichero.

Cómo se lee el resultado

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Estrategias

Puerta IS/OOS

Generación

Datos

Estudios

Carteras

Generación

INSTALL custodian PROYECTO USDJPY_emaCross_H1

CORRIENDO tarea MCR 7 OHLC · 0 / 10 estrategias · por estrategia · la tarea lleva 26 s · workflow de hoy: 25 min 13 s · log hace 14 s

LAS TAREAS DEL PROYECTO, EN EL ORDEN EN QUE SQX LAS CORRE

#	tarea	tipo	en este start	estado	hechas / total	tiempo	por estrategia	databank de salida	en disco
1	CONSTRUCCION	Build	-	hecha antes	- / -	1 min 14 s	-	Results	100
2	OOS	Retest	-	hecha antes	100 / 100	6 s	12 ms	OOS	40
3	Retest Markets - Family	Retest	-	hecha antes	40 / 40	1 min 24 s	282 ms	Retest Markets - Family	10
4	MCR 1 Bar	Retest	si	hecha	10 / 10	23 s	1.0 s	MCR 1 Bar	0
5	MCR 2 Spread	Retest	si	hecha	10 / 10	3 min 20 s	16.0 s	MCR 2 Spread	0
6	MCR 3 Slippage	Retest	si	hecha	10 / 10	3 min 24 s	16.0 s	MCR 3 Slippage	0
7	MCR 4 MinDist	Retest	-	saltada	-	-	-	MCR 4 MinDist	0
8	MCR 5 Params	Retest	si	hecha	10 / 10	16 s	1.0 s	MCR 5 Params	0
9	MCR 6 Exits	Retest	si	hecha	10 / 10	3 min 27 s	16.0 s	MCR 6 Exits	0
10	MCR 7 OHLC	Retest	si	en curso	0 / 10	26 s	-	MCR 7 OHLC	0
11	MCR 8 Stress	Retest	si	en cola	-	-	-	MCR 8 Stress	0
12	SPP IS	Retest	-	inactiva	-	-	-	SPP IS	0
13	SPP OOS	Retest	-	inactiva	-	-	-	SPP OOS	0
14	WFM	Retest	-	inactiva	-	-	-	WFM	0
15	CrossTF	Retest	-	inactiva	40 / 40	20 s	46 ms	CrossTF	40
16	WFC 1 IS	Retest	-	inactiva	-	-	-	WFC_Build	0
17	WFC 2 00S1	Retest	-	inactiva	-	-	-	WFC_00S1	0
18	WFC 3 00S2	Retest	-	inactiva	-	-	-	WFC_00S2	0

EL LOG DEL INSTALL, ÚLTIMAS LÍNEAS DE PROGRESO

21:22:10.577 MCR 6 Exits : Starting strategies retesting...
21:22:10.580 MCR 6 Exits : Initializing backtest data...
21:22:10.580 MCR 6 Exits : Loading backtest data for Main test - USDJPY_DukasM1_the5ers / H1
21:22:10.580 MCR 6 Exits : Data loaded from memory
21:22:11.837 MCR 6 Exits : All backtest data prepared
21:22:11.838 MCR 6 Exits :
21:25:37.313 MCR 6 Exits : Task finished in 206.73 s.
21:25:37.387 MCR 7 OHLC :
21:25:37.387 MCR 7 OHLC : Starting strategies retesting...
21:25:37.391 MCR 7 OHLC : Initializing backtest data...
21:25:37.391 MCR 7 OHLC : Loading backtest data for Main test - USDJPY_DukasM1_the5ers / H1
21:25:37.391 MCR 7 OHLC : Data loaded from memory
21:25:38.665 MCR 7 OHLC : All backtest data prepared
21:25:38.665 MCR 7 OHLC :

Recargar

registry.csv · runs.csv · library/ — 7 filas x 6 timeframes · daemon en 127.0.0.1

Cada start de un proyecto es un **paso del workflow**: stage deja activa solo la tarea del paso y SQX marca las demás como SKIPPED, inactive task . Por eso los estados son estos:

estado	qué significa
en curso	la última tarea que escribe en el log; en negrita, y con su porcentaje arriba cuando SQX lo da
hecha	el log vio <code>Task finished</code> en este start
hecha antes	inactiva en este start, pero su databank tiene estrategias: la hizo un start anterior
en cola	activa en este start y aún no empezada
saltada	inactiva en este start y con el databank vacío
inactiva	apagada, sin start en marcha

La columna «en disco» son ficheros: SQX los escribe al sincronizar, no según los produce. En la captura, el custodio va por las tareas del MC Retest sobre las 20 estrategias que dejó el retest de mercados: cada una tarda entre 1 y 16 segundos según la perturbación.

Un ejemplo completo

1. Otra sesión lanza el paso 13 en el custodio con `/mcretest`.
2. `bin/algoui --zone "Generación"`, install `custodian`, proyecto `USDJPY_emaCross_H1`.
3. Arriba: `CORRIENDO · tarea MCR 3 Slippage · 7 / 20 estrategias · 16,0 s por estrategia · la tarea lleva 1 min 52 s · workflow de hoy: 18 min`. En la tabla, `MCR 1 Bar` y `MCR 2 Spread` en verde como hechas con su `10 / 20` y su tiempo, `MCR 3` en negrita, `MCR 4` a `MCR 8` en cola. Debajo, el log diciendo «Loading backtest data for Main test».
4. Cuando la línea pase a `terminado`, los ocho databanks MCR tendrán su recuento.

Qué NO te dice

- **Cuánto falta, con exactitud.** Hechas × tiempo por estrategia es una estimación: las estrategias no tardan lo mismo, y un build no tiene total conocido. El porcentaje del nombre del hilo solo lo escriben las tareas walk-forward.
- **De quién es el start.** El log nombra el proyecto solo cuando el start llega por la API (`Starting project '...'`); un start desde la GUI del maestro no se atribuye hasta que guarda un databank (`knowhow/locations/log-names-project-only-via-cli.md`).
- **Si el resultado es bueno.** Solo cuenta ficheros.

Si algo falla

- **«sin log de hoy»:** el install no ha escrito nada hoy; o no ha corrido nada o el log está en otra fecha. La zona solo lee el fichero del día.
- **«otro proyecto corre»:** no es un error. El install está ocupado con el proyecto que nombra.
- **El recuento no sube aunque corre:** normal hasta que SQX sincroniza el databank.

44. La aplicación de escritorio — estrategias

La tercera zona de la ventana, y la primera que lleva el aspecto nuevo: fondo casi negro sin tinte, letra monoespaciada para toda cifra y todo nombre, filas finas separadas por líneas en vez de tarjetas. Se elige **un proyecto**, y debajo salen **sus databanks ya exportados en AlgoData**, las estrategias de cada uno, y al pulsar una, **qué dijo ya cada módulo de análisis sobre ella** — con un botón **correr** en los módulos que todavía no han hablado y pueden hacerlo desde aquí.

Qué pregunta responde

- **¿Qué databanks de este proyecto tengo ya en Python?** Arriba a la izquierda se elige el proyecto; debajo, el activo que opera, leído de su nombre (si el nombre no lo dice, se elige a mano: los módulos lo necesitan para el feed y los costes). La lista son los databanks que `export` sacó a `metrics/` y las cosechas de `harvest/`, con su recuento y la fecha. Un databank en amarillo no tiene `manifest.json`: nadie firmó ese export y el auditor lo marca.
- **¿Qué estrategias hay en este databank, y cuáles pintan bien?** Una fila por estrategia con beneficio neto, operaciones, profit factor, Sharpe, Ret/DD y drawdown máximo, en IS y en OOS cuando el export los trae. Se ordena por cualquier columna y se filtra por nombre. Los negativos en rojo.
- **¿Qué se sabe ya de esta estrategia?** Un bloque por módulo de análisis, en el orden del workflow: puerta IS/OOS, curado, cross-market, MC Retest, Monte Carlo, nulo de entrada, forma del beneficio, calidad de la entrada, exposición, WFC, WFM y decaimiento. Cada bloque dice en qué databank, qué día y qué fichero dejó el módulo su veredicto sobre esa estrategia, y lo enseña con sus cifras. Si la estrategia entró en el pipeline, las etapas del ledger con su porcentaje y su línea de estado van arriba.
- **¿Qué puedo correrle ahora mismo?** El botón **correr** de cada bloque lanza ese módulo sobre este databank. Sale solo cuando sus entradas existen: un export de operaciones de un solo mercado, la cosecha, el export cross-market. Si no, el bloque dice qué falta y con qué skill se consigue.

Cuándo lo usas, y cuándo no

Lo usas cuando vuelves a un proyecto y no recuerdas qué pruebas se le hicieron a una estrategia; cuando quieres cruzar el veredicto de dos módulos sin abrir dos paneles; cuando vas a lanzar un análisis y quieres saber si ya se hizo; y para copiar el comando exacto de lo que falta.

No lo usas para nada que toque SQX. La ventana **no arranca SQX ni un worker**: lo que corre son los módulos de Python sobre lo que ya está exportado. Tampoco lee el databank vivo de SQX: si no has exportado, no aparece. Y no hace curado (`/curate`) ni WFC (va por el pipeline).

Antes de empezar

- Algún databank exportado: `python3 -m sqx.export.export_metrics --project X --databank Y` (manual 04-export.md) o una cosecha de `studies.screening.gate.harvest` (manual 31-puerta-oos.md).
- La app abierta con `bin/algoui`. **Si ya la tenías abierta antes de este cambio, ciérrala y vuelve a abrirla**: la ventana comparte un demonio con las otras ventanas y el demonio viejo no conoce las rutas nuevas.

Cómo se ejecuta

```
bin/algoui
```

Y en la barra lateral, **Estrategias**. O directamente desde el escritorio: el acceso **AlgoProject Estrategias** (`~/Desktop/AlgoProject-Estrategias.desktop` , copia de `~/.local/share/applications/algoproject-estrategias.desktop`) abre la misma ventana ya en esta zona, con `bin/algoui --zone Estrategias` . Es el único flag: el nombre de la zona tal y como sale en la barra lateral.

Pulsar **correr** lanza el módulo en segundo plano como hijo del demonio, con los mismos argumentos que pondrías a mano (el tooltip del botón los enseña). Mientras corre, el bloque lo dice en ámbar; al terminar, verde con código 0 o rojo con el código de error, y debajo las últimas 25 líneas de su salida. Los módulos de una estrategia tardan segundos; Monte Carlo, MC Retest o WFM sobre el databank entero, minutos. Se pueden lanzar varios a la vez. Todo es lectura de disco en el demonio: abrir un databank de 10.000 filas tarda una décima de segundo, y la ficha de una estrategia lee todos los CSV de informes del proyecto, decenas de ficheros pequeños, en menos de otra décima. No toca SQX: se puede usar con el maestro abierto y con un worker construyendo.

Qué produce

Un log por cada botón pulsado, en `~/Desktop/AlgoData/logs/ui/<fecha-hora>-<módulo>.log` , con la salida entera del módulo. Lo que el módulo escriba —su `reports/...` — es cosa suya y está en su página del manual. La ventana no escribe nada más. El botón «abrir informe html» abre en el navegador la página que un módulo ya escribió para esa estrategia (desde el 25-09 todos los estudios la escriben, en `reports/<proyecto>/<databank>/<día>/<estudio>/estrategias/<nombre>.html` , al lado de su `.json`).

Cómo se lee el resultado

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Estrategias

Puerta IS/OOS

Datos

Generación

Estudios

Carteras

Recargar

Estrategias13 databanks en 5 proyectos · 12 módulos leídos

PROYECTO

XAUUSD

ACTIVO

XAUUSD

DATABANKS EXPORTADOS

005_10000

005-Rexpect10000

005-Rexpect210000

005-Rsquared10000

005-Sharpe9997

SPP_005165

XAUUSD / OOS

17.9.39

estrategia	Net profit · IS	Net profit · OOS	# of trades
Strategy 17.9.39	-24,620	7,737	

registry.csv · runs.csv · library/ — 7 filas x 6 timeframes · daemon en 127.0.0.1

ESTRATEGIA

Strategy 17.9.39

2 de 12 módulos con resultado · 0 páginas · en el pipel

PIPELINE · ETAPA VERDICT

sppultra100% -> /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/SPP_IS/2026-09-22

design100% -> /home/sergioguslw/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39/plan.csv

build100% -> /home/sergioguslw/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39

ran100% 2000 reteseadas, panel en retest.csv

collectraw100%

wfc100% -> /home/sergioguslw/Desktop/AlgoData/pipeline/XAUUSD/Strategy_17-9-39/wfc.html

verdict100% reject, falla wfc.rho

PUERTA IS/OOS paso 8

necesita la cosecha de este databank (studies.screening.gate.harvest, skill /oos-gate)

SPP IS · 2026-09-23 · gate/verdict_build.csv

verdictMANTENER

identityd10dfa86099efbcbfabd9720e0b8cb7fab158e6..

SPP IS · 2026-09-23 · gate/verdict.csv

verdictMANTENER

identityd10dfa86099efbcbfabd9720e0b8cb7fab158e6..

CURADO DEL DATABANK paso 8 → 9

es una skill: /curate, desde Claude Code

sin resultado

CROSS-MARKET paso 10

necesita el export del retest cross-market (skill /crossmarket)

sin resultado

MC RETEST paso 14

sin resultado

correr

MONTE CARLO DE OPERACIONES paso lectura extra

necesita un export raw de operaciones de este databank (skill /export)

sin resultado

NULO DE ENTRADA paso lectura extra

necesita un export raw de operaciones de este databank (skill /export)

Tres columnas. A la izquierda los databanks por proyecto, con el número de estrategias; «cosecha» marca los que vienen de harvest/ . En el centro, la tabla del databank abierto. A la derecha, la ficha de la estrategia seleccionada:

- La línea bajo el nombre resume cuántos módulos tienen algo, cuántas páginas html hay y si la estrategia está en el pipeline.
- **Pipeline:** cada etapa del ledger con su porcentaje y su última línea de estado. Verde si terminó, ámbar si está a medias.
- **Un bloque por módulo.** La primera línea gris dice dónde está el resultado: databank, día y fichero. Si el fichero tiene varias filas para la estrategia (un WFC tiene una por punto de parámetro) se enseña la primera y se dice cuántas hay. La palabra del veredicto va en color: verde MANTENER / worth_it , rojo DESCARTAR / FAIL , ámbar DUDOSA / MARGINAL , gris cualquier palabra que ningún módulo de este proyecto use — se enseña, no se esconde.
- Debajo, cada cifra de la fila del informe con su nombre tal y como el módulo lo escribió. La ventana no las interpreta: la página de cada módulo en este manual dice qué es buena y qué mala.

AlgoProject

Cobertura
Plantillas
Nueva plantilla
Paletas
Activos
Estrategias
Puerta IS/OOS
Datos
Generación
Estudios
Carteras

Estrategias
13 databanks en 5 proyectos · 12 módulos leídos

PROYECTO

TestUSDJPY_Workflow_v1

ACTIVO

USDJPY

DATABANKS EXPORTADOS

MCR_0_Stress 4
005 17
Results · cosecha 17
Retest_Markets_-_Family

TESTUSDJPY_WORKFLOW_V1 / RETEST_MARKETS_-_FAMILY

6.1.81

estrategia	Net profit · IS	Net profit · OOS	# of trades · IS	P
Strategy 6.1.81	34,774	0	638	

ESTRATEGIA

Strategy 6.1.81

3 de 12 módulos con resultado · 0 páginas · fue:

PUERTA IS/OOS

paso 8

necesita la cosecha de este databank (studies.screening.gate.harvest, skill /oos-gate)

Results · 2026-09-25 · gate/verdict build.csv

verdict MANTENER

identity 84149f0f135ae86208bd27269ddf6cc77276e:

Results · 2026-09-25 · gate/verdict.csv

verdict MANTENER

identity 84149f0f135ae86208bd27269ddf6cc77276e:

Results · 2026-09-24 · gate/verdict build.csv

verdict MANTENER

identity 84149f0f135ae86208bd27269ddf6cc77276e:

y 1 resultados más antiguos

CURADO DEL DATABANK

paso 8 → 9

es una skill: /curate, desde Claude Code

Retest Markets - Family · 2026-09-24 · curate/verdict-125836.csv

verdict MANTENER

identity 84149f0f135ae86208bd27269ddf6cc77276e:

Retest Markets - Family · 2026-09-24 · curate/verdict-125812.csv

verdict MANTENER

identity 84149f0f135ae86208bd27269ddf6cc77276e:

Retest Markets - Family · 2026-09-24 · curate/verdict-125136.csv

verdict MANTENER

identity 84149f0f135ae86208bd27269ddf6cc77276e:

y 6 resultados más antiguos

CROSS-MARKET

paso 10

correr otra vez

Retest Markets - Family · 2026-09-24 · crossmarket/verdict.csv

verdict DESCARTAR reason solo 0 de 9 mercados con la

markets	9	cleared	0
fraction	0	under alpha	0
paired under	0	edge r	-0.02849
worst pf	0.8454	pf cv	0.0774
family	entry+exit	missing	0
warnings	24		

MC RETEST

paso 14

correr

Recargar

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

Una estrategia de `TestUSDJPY_Workflow_v1` sobre su export cross-market. Cross-market ofrece «correr otra vez»; MC Retest acaba de terminar desde el botón y enseña el final de su salida; Monte Carlo, nulo y exposición explican que ese export mezcla mercados y no les vale.

Un ejemplo completo

1. `bin/algoui`, **Estrategias**. Bajo `XAUUSD`, pulsar `OOS 10000`.
2. En «filtrar por nombre» escribir `17.9.39` y pulsar la fila.
3. La ficha dice: en el pipeline hasta `verdict`, y la puerta la mantuvo en `SPP_IS` el 2026-09-23; el WFC tiene sus puntos en `Seq._0pt._IS/2026-09-19`. Cross-market, MC Retest, Monte Carlo y WFM: sin resultado, con su comando debajo.
4. Pulsar **correr** en MC Retest. El bloque pasa a «en curso»; al minuto, «terminó» y las últimas líneas de la tabla; la ficha se redibuja sola con el veredicto en su sitio.

Qué NO te dice

- **No dice si la estrategia es buena.** Enseña cifras y palabras que otros módulos decidieron. Las reglas de cada uno están en su página.
- **No sabe nada de un informe que no nombre a la estrategia en una columna `strategy`.** Es la convención que todos los módulos siguen hoy (`knowhow/locations/report-csv-conventions.md`); un informe que la rompa no aparece, sin aviso.
- **No enseña las estrategias que están en SQX y no exportadas.** Un databank vivo con 5.000 estrategias y ningún export es invisible aquí.
- **Un veredicto viejo sigue apareciendo.** Si el módulo se corrió varias veces, se enseñan las tres más recientes y se dice cuántas más hay. La ventana no decide cuál vale.

- **El botón no sabe si el módulo tiene sentido aquí.** Comprueba que las entradas existan, no que el databank sea el adecuado: MC Retest sobre un proyecto sin tareas MCR termina con error, y el error es lo que se enseña.

Si algo falla

- **La zona sale vacía o la pestaña no existe:** el demonio que atiende la ventana es anterior a este cambio. Cerrar todas las ventanas de la app y volver a abrir; si sigue, `pgrep -af ui.daemon.serve` y matar ese PID.
- **Un botón termina en rojo:** el final de su log está debajo; el log entero en `AlgoData/logs/ui/`. Casi siempre es una entrada que no era la que el módulo esperaba.
- **KeyError: 'strategy_build'** al abrir una cosecha: el `metrics.parquet` es de antes del 2026-09-23, cuando la columna se llamaba de otra forma. Rehacer la cosecha con `studies.screening.gate.harvest`.

45. La aplicación de escritorio — la puerta IS/OOS

La zona del **paso 8** del workflow: el primer análisis dentro y fuera de muestra, el que pasa de miles de estrategias a decenas. Enseña lo que la puerta (`studies.screening.gate.report` , manual `05-cribado-oos.pdf` > 29-puerta) hizo con una cosecha, criba a criba y estrategia a estrategia, y la lanza con los umbrales a la vista. Es el primer panel construido a partir del catálogo `docs/AgentPDFs/catalogo-para-la-ui-2026-09-25.md` (sección 3.2); el resto del catálogo llegará como paneles al lado de este.

Qué pregunta responde

- **¿Qué cosechas hay y cuáles están juzgadas?** El desplegable de arriba lista cada cosecha de `AlgoData/harvest/` con la fecha de su puerta y el resultado (`120 → 45`), o «sin puerta todavía». Debajo, lo que la cosecha juntó: cuántas estrategias tenía el build, cuántas dejó SQX en el retest, cuántas emparejó por identidad y cuántas se quedaron sin retest.
- **¿Cuánta población mata cada criba?** El **embudo**: una barra por criba, a escala, verde lo que pasa y rojo lo que muere; gris las cribas *soft*, que miden y no eliminan. Es la cifra que más se olvida y aquí se lee de un vistazo. Al pasar el ratón, el porqué de la criba y sus umbrales.
- **¿Qué le pasó a cada estrategia?** El **scorecard**: una fila por estrategia con su identidad, su nombre en cada databank, en qué criba murió, si sobrevive, y el valor que cada criba midió en ella, en verde o en rojo. Se ordena por cualquier columna; «solo supervivientes» esconde a las muertas. Al pasar el ratón por un valor, la nota de la criba.
- **¿Cómo es esta estrategia dentro y fuera?** Al pulsar una fila: la curva diaria de los dos backtests (el retest empalmado al último nivel del build, como hace la puerta al pegar por retornos), diez métricas IS contra OOS lado a lado, y cada criba con su valor y su nota.

Cuándo lo usas, y cuándo no

Lo usas justo después de una cosecha, para ver cuánto mata cada criba antes de decidir nada; para buscar en qué criba cae una estrategia concreta; y para probar un umbral distinto sin tocar `studies/screening/gate/config.yaml` : se escribe en la caja de umbrales y queda en el manifest del informe.

No lo usas para hacer la cosecha: eso es `studies.screening.gate.harvest` y toca el conductor (skill `/oos-gate`), y la ventana no arranca SQX. Tampoco aplica el veredicto al databank: eso es `/curate` . Y no decide qué umbrales son los buenos: los del fichero son laxos a propósito (decisión del dueño, 2026-09-23) y están registrados en `ledger/thresholds.yaml` .

Antes de empezar

- Una cosecha en `AlgoData/harvest/<proyecto>/<databank>/<día>/` : `python3 -m studies.screening.gate.harvest --project P --databank Results --oos-databank OOS` , manual `05-cribado-oos.pdf` > 29-puerta .
- El activo del proyecto en `assets/symbols/` con su `sqx_symbol` : la puerta necesita el feed para las barras del mono.
- La app abierta con `bin/algoui` , o `bin/algoui --zone "Puerta IS/OOS"` .

Cómo se ejecuta

En la barra lateral, **Puerta IS/OOS**. Elegir la cosecha en el desplegable. Si ya tiene puerta, se pinta todo; si no, el botón **correr la puerta** la lanza en segundo plano como hijo del demonio. Sobre 120 estrategias tarda segundos; el mono con 2.000 corridas por estrategia es lo que manda, y no toca SQX. Al terminar, el estado junto al botón pasa a «terminó» (o al código de error, con el final de la salida en su tooltip) y la zona se recarga sola.

control	qué hace
cosecha	qué cosecha se mira. Una cosecha juzgada varias veces enseña su puerta más reciente
activo	el símbolo cuyo feed leen las barras del mono. Se rellena desde el informe, o desde el nombre del proyecto
umbrales	overrides <code>criba.umbral=valor</code> separados por espacios, como <code>--set</code> . Vacío = <code>studies/screening/gate/config.yaml</code>
solo supervivientes	esconde las filas muertas del scorecard

Qué produce

Lo que produce `studies.screening.gate.report`, en `AlgoData/reports/<proyecto>/<databank>/<día de hoy>/gate/`: `scorecard.parquet`, `funnel.csv`, `verdict.csv`, `verdict_build.csv`, `gate.md/.html/.json` y su manifest, que apunta a la cosecha que juzgó y guarda los overrides. Un informe se fecha el día que corre, no el de la cosecha; la ventana los une por ese manifest. Más el log del botón en `AlgoData/logs/ui/`.

Cómo se lee el resultado

AlgoProject

Cobertura

Plantillas

Nueva plantilla

Paletas

Activos

Estrategias

Puerta IS/OOS

Datos

Generación

Estudios

Carteras

Puerta IS/OOS

COSECHA

TestUSDJPY_Workflow_v1 / Results · 2026-09-24 · puerta del

build 50 · retest 17 (005) · emparejadas 17 · sin retest 33 · 21,769 operaciones

ACTIVO

USDJPY

UMBRALES

criba.umbral=valor, separados por espacios · vacío = los de studies/screening/gate/config.yaml

correr la puerta

solo supervivientes

EMBUDO · ENTRAN → PASAN

presencia

sanidad

estaticas

degradacion

forma

mono

familia

redundancia

50 → 17 → 33

17 → 17

17 → 17

17 → 17

17 → 17

17 → 17

17 → 0 soft

17 → 17 soft

LAS CRIBAS, EN ORDEN · PASA EL RATÓN PARA EL PORQUÉ

ventana 005 2017-11-01 - 2022-12-29 · informe del 2026-09-25

presencia (hard): -

sanidad (hard): min trades=20

estaticas (hard): keep= 'Net profit [005]' > 0, value=Net profit [005]

degradacion (hard): min retention=0.0, min t=0.0, min years positive=1, max concentration=1.0

forma (hard): max dd ratio=5.0

mono (hard): statistic=sharpe, rung=timing, max p=0.5

familia (soft): alpha=0.05

redundancia (soft): -

SCORECARD · UNA FILA POR ESTRATEGIA, UNA COLUMNA POR CRIBA

estrategia

muere en

sobrevive

presencia

Strategy 11.1.63

Strategy 6.1.77

Strategy 11.1.50

Strategy 6.1.69

Strategy 6.1.57

Strategy 6.1.79

Strategy 6.1.46

Strategy 6.1.81

Strategy 11.1.57

Strategy 6.1.71

Strategy 6.1.60

Strategy 23.1.71

Strategy 23.1.60

Strategy 23.1.53

Strategy 23.1.46

Strategy 23.1.66

Strategy 23.1.55

Strategy 23.1.69 (SQX la ..

Strategy 6.1.49 (SQX la ..

Strategy 23.1.70 (SQX la ..

Strategy 23.1.67 (SQX la ..

Strategy 6.1.84 (SQX la ..

Strategy 6.1.78 (SQX la ..

Strategy 6.1.45 (SQX la ..

Strategy 6.1.67 (SQX la ..

Strategy 11.1.49 (SQX la ..

Strategy 23.1.48 (SQX la ..

Strategy 6.1.52 (SQX la ..

Strategy 23.1.59 (SQX la ..

Strategy 6.1.65 (SQX la ..

Strategy 6.1.64 (SQX la ..

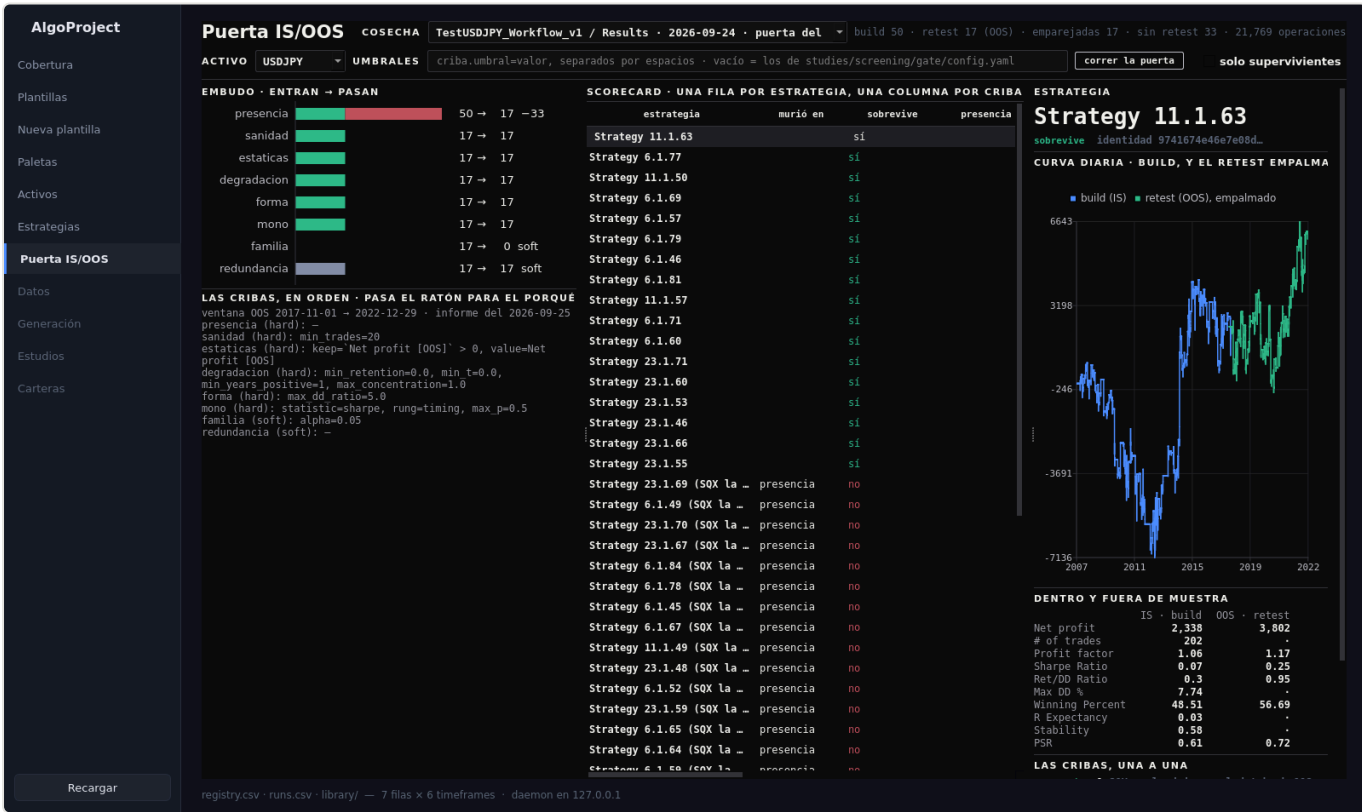
Strategy 6.1.60 (SQX la ..

Elige una estrategia en el scorecard.

registry.csv · runs.csv · library/ — 7 filas × 6 timeframes · daemon en 127.0.0.1

Recargar

Arriba, la cosecha y sus cuentas. A la izquierda, el embudo y debajo la lista de cribas con sus umbrales y la ventana OOS del informe. En el centro, el scorecard. Un `murió en vacío con sobrevive = no` no existe: toda muerta nombra su criba. Una estrategia con `– (SQX la tiró)` como nombre murió en `presencia`: el retest no la tiene, y por eso no tiene nombre allí.



La ficha. La curva azul es el build, la verde el retest empalmado. Las métricas en dos columnas: un `·` es una métrica que ese lado no exporta (la vista de SQX emite menos columnas OOS que IS). Las cribas: verde pasa, rojo no pasa, gris soft con `✓/X` informativo, y `«no llegó»` si murió antes.

Un ejemplo completo

1. `bin/algoui --zone "Puerta IS/OOS"`. En cosecha, `TestUSDJPY_Workflow_v1 / Results · 2026-09-24`.
2. El embudo dice: 50 entran, 33 mueren en `presencia` (SQX no las dejó en el retest), y las 17 restantes pasan todo. Con los umbrales laxos de hoy, la puerta solo tumba lo que SQX ya tumbó.
3. Escribir en umbrales `sanidad.min_trades=200 forma.max_dd_ratio=1.5` y pulsar **correr la puerta**. A los segundos, «terminó», y el desplegable muestra la puerta del día con el nuevo resultado; el embudo enseña dónde cayeron las que ahora mueren, y la línea de cribas lleva los overrides.
4. Pulsar una fila: la curva dice si el retest siguió subiendo o se aplanó, y las cribas de la derecha dicen con qué número.

Qué NO te dice

- **No dice si un umbral es el bueno.** Enseña cuánta población mata; decidir el umbral es el paso que viene después, con la distribución delante, y se anota en `ledger/thresholds.yaml`.

- **No puede deshacer una selección previa.** Si el build ya filtró con el periodo OOS, todas las cribas leen inertes (`studies/screening/gate/README.md`). El embudo lo delata: nadie muere salvo en presencia.
- **No es el veredicto aplicado.** Hasta `/curate` , el databank de SQX sigue con todas.
- **La curva empalmada no es una cuenta real.** Cada lado fue su propio backtest desde su saldo; el empalme es para leer la forma, no el beneficio total.

Si algo falla

- **El desplegable está vacío:** no hay ninguna cosecha en `AlgoData/harvest/` .
- **«falló» tras correr:** el tooltip del estado lleva las últimas líneas; el log entero en `AlgoData/logs/ui/` . Lo habitual es un `--set` mal escrito (la criba o el umbral no existen) o un feed sin barras en `AlgoData/bars/` .
- **El activo sale mal:** el nombre del proyecto no contiene ningún símbolo y no había informe previo. Elígelo a mano antes de correr.